

cheatsheet_openmp

Cheat Sheet — OpenMP (Programação Concorrente)

Referência de consulta rápida. Organizado por: Regiões Paralelas → Worksharing → Escopo de Dados → Schedule → Sincronização → Tasks → Funções de Biblioteca / Variáveis de Ambiente.

1. Modelo Fork-Join

O programa começa com **1 thread (master)**. Ao encontrar `#pragma omp parallel`, o runtime cria um **time de threads** (fork) que executam o mesmo bloco de código. Ao final do bloco, há uma **barreira implícita**: todas esperam, e voltam a ser 1 thread só (join).

1 thread —[fork: parallel]→ N threads executam o bloco —[join]→ 1 thread

2. Diretivas de Região Paralela

Diretiva	O que faz	Barreira implícita?
<code>#pragma omp parallel</code>	Cria o time de threads; todo o bloco é replicado e executado por cada thread	Sim, no fim do bloco
<code>#pragma omp parallel for</code>	Forma combinada: cria o time e distribui as iterações do laço seguinte entre as threads	Sim, no fim do laço
<code>#pragma omp parallel sections</code>	Forma combinada de <code>parallel</code> + <code>sections</code> (distribui blocos de código distintos entre threads)	Sim

Cláusulas comuns de `parallel` :

Cláusula	Efeito
<code>num_threads(N)</code>	Define quantas threads o time terá (sobrepõe <code>OMP_NUM_THREADS</code> para essa região)
<code>if(expr)</code>	Se <code>expr</code> for falsa, a região roda sequencialmente (só a thread master)

Cláusula	Efeito
<code>default(shared none)</code>	Define o escopo padrão de variáveis não listadas explicitamente. <code>none</code> obriga você a escopar tudo manualmente (boa prática!)
<code>shared(list)</code> / <code>private(list)</code> / <code>firstprivate(list)</code>	Ver seção 4

3. Diretivas de Worksharing (divisão de trabalho)

Diretivas que **dividem** trabalho entre as threads de um time já criado (usadas dentro de `#pragma omp parallel`).

Diretiva	O que faz	Barreira implícita?
<code>#pragma omp for</code>	Divide as iterações de um laço <code>for</code> entre as threads do time (cada iteração roda uma vez só , por uma thread)	Sim (a menos que use <code>nowait</code>)
<code>#pragma omp single</code>	Apenas uma thread do time (a primeira a chegar, não necessariamente a 0) executa o bloco	Sim (a menos que use <code>nowait</code>)
<code>#pragma omp master</code>	Apenas a thread 0 (master) executa o bloco. As demais não esperam — seguem direto	Não
<code>#pragma omp sections</code> / <code>#pragma omp section</code>	Divide blocos de código distintos entre as threads (cada <code>section</code> roda em uma thread diferente)	Sim, no fim de <code>sections</code>

Analogia: `master` = "só o anfitrião compra o bolo, ninguém espera"; `single` = "quem chegar primeiro compra o bolo, todo mundo espera essa pessoa voltar".

Cláusulas de `for` / `parallel for`:

Cláusula	Efeito
<code>schedule(tipo[, chunk])</code>	Como as iterações são distribuídas — ver seção 5
<code>collapse(n)</code>	Colapsa <code>n</code> laços aninhados em um único espaço de iteração antes de dividir entre threads (melhora balanceamento em laços aninhados)
<code>ordered</code>	Habilita o uso da construção <code>#pragma omp ordered</code> dentro do laço

Cláusula	Efeito
<code>nowait</code>	Remove a barreira implícita do fim do <code>for / single / sections</code>
<code>reduction(op:var)</code>	Ver seção 6
<code>private</code> , <code>firstprivate</code> , <code>lastprivate</code>	Ver seção 4

4. Escopo de Dados (Data Scoping)

Regra padrão: variáveis declaradas **fora** da região paralela são `shared` por padrão — **exceto** a variável de controle de um `for` paralelizado, que é `private` automaticamente.

Cláusula	Comportamento
<code>shared(list)</code>	Todas as threads leem/escrevem na mesma posição de memória. Sem proteção = risco de condição de corrida (race condition)
<code>private(list)</code>	Cada thread recebe uma cópia própria e não-inicializada . O valor de fora não é herdado, e ao sair da região, o valor original de fora permanece indefinido/inalterado
<code>firstprivate(list)</code>	Como <code>private</code> , mas a cópia de cada thread é inicializada com o valor que a variável tinha antes da região começar
<code>lastprivate(list)</code>	Como <code>private</code> , mas ao final do laço/seção, o valor da última iteração/seção logicamente executada é copiado de volta para a variável original
<code>default(shared\ none)</code>	Define o escopo padrão das variáveis não listadas. <code>none</code> força escopar tudo manualmente (evita erros silenciosos)

Por que `shared` sem proteção é perigoso: uma linha como `resultado = resultado + tid` não é uma operação atômica — envolve **ler**, **somar** e **escrever** de volta na memória, em passos separados. Se duas threads fazem isso ao mesmo tempo na mesma variável, uma pode sobrescrever o resultado da outra → **condição de corrida**, resultado imprevisível.

5. Cláusula `schedule` (Escalonamento de Laços)

Tipo	Quando decide a divisão	Comportamento	Melhor para
<code>static[, chunk]</code>	Antes de rodar (compile/início)	Divide em blocos fixos e previsíveis. Sem <code>chunk</code> : blocos de tamanho $\approx N/\text{threads}$. Com <code>chunk</code> : distribui em rodízio (round-robin) de <code>chunk</code> em <code>chunk</code>	Iterações com custo uniforme — menor overhead
<code>dynamic[, chunk]</code>	Durante a execução	Cada thread pega um pedaço (chunk, padrão=1); ao terminar, pede o próximo pedaço disponível na fila	Iterações com custo desigual/imprevisível — mas mais overhead de coordenação
<code>guided[, chunk]</code>	Híbrido	Começa com pedaços grandes , diminuindo progressivamente até o mínimo <code>chunk</code>	Meio-termo: carga desigual, mas com menos overhead que <code>dynamic</code> puro
<code>auto</code>	Implementação decide	Deixa o runtime/compilador escolher a melhor estratégia	Quando você não sabe qual usar
<code>runtime</code>	Variável de ambiente <code>OMP_SCHEDULE</code>	Usa o que estiver definido em <code>OMP_SCHEDULE</code> em tempo de execução, sem recompilar	Testes/tuning sem recompilar

⚠ **Importante:** `schedule` controla **quais** iterações cada thread executa, mas **nunca** garante a ordem de saída (ex: prints no terminal), a menos que se use `ordered`.

6. Sincronização

Diretiva	O que faz
<code>#pragma omp critical[(nome)]</code>	Só uma thread por vez pode executar o bloco (região de exclusão mútua). Se usar o mesmo <code>nome</code> em vários blocos, eles competem pelo mesmo "cadeado"; nomes diferentes = cadeados independentes
<code>#pragma omp atomic [read\ write\ update\ capture]</code>	Protege uma única operação de memória simples (mais leve que <code>critical</code>) <code>read</code> : leitura atômica

Diretiva	O que faz
	• <code>write</code> : escrita atômica • <code>update</code> (padrão): operação tipo <code>x++</code> , <code>x += y</code> • <code>capture</code> : lê e atualiza numa única operação atômica (ex: <code>z = x++</code>)
<code>#pragma omp barrier</code>	Todas as threads do time esperam até que todas cheguem nesse ponto antes de continuar
<code>#pragma omp ordered</code>	Dentro de um <code>for</code> com cláusula <code>ordered</code> , força que aquele trecho específico execute na mesma ordem sequencial do laço original
<code>nowait</code> (cláusula)	Remove a barreira implícita do fim de <code>for</code> , <code>single</code> ou <code>sections</code> — usar com cuidado, pois pode gerar condições de corrida se as threads seguintes dependerem do resultado
<code>reduction(operador:var)</code>	Cada thread mantém uma cópia privada da variável, acumula seu resultado parcial, e no final o runtime combina tudo com o <code>operador</code> (ex: <code>+</code> , <code>*</code> , <code>max</code> , <code>min</code>) de forma segura, sem condição de corrida

`critical` **vs** `atomic`: use `atomic` quando a operação é simples (uma leitura/escrita/incremento de uma variável escalar) — é mais rápido. Use `critical` quando o bloco protegido tem **múltiplas instruções** ou lógica mais complexa.

7. Tasks (Tarefas)

Tasks são **unidades independentes de trabalho** (código + ambiente de dados + variáveis de controle), úteis quando o padrão de paralelismo não é um laço regular (ex: recursão, listas encadeadas, grafos de dependência).

7.1 Criação de tarefas

Construção	Quando cria tarefa(s)
Início de região <code>parallel</code>	Cria tarefas implícitas (uma por thread)
<code>#pragma omp task</code>	Cria uma tarefa explícita
<code>#pragma omp taskloop</code>	Cria tarefas explícitas, uma para cada bloco (chunk) do laço
<code>#pragma omp target</code>	Cria uma tarefa target (para dispositivos, ex: GPU)

O **runtime decide quando** as tarefas realmente executam — podem ser adiadas (colocadas num "pool de tarefas") ou executadas imediatamente. A thread que executa a tarefa pode ser **diferente** da thread que a criou.

7.2 Cláusulas de `#pragma omp task`

Cláusula	Efeito
<code>if(expr)</code>	Se falsa, a tarefa é executada de forma sequencial e imediata (undeferred) pela thread que a encontrou
<code>final(expr)</code>	Se verdadeira, a tarefa não terá tarefas-filhas (força execução sequencial dos descendentes) — útil para "cortar" recursão excessivamente fina
<code>mergeable</code>	Permite que o ambiente de dados da tarefa seja mesclado com o da tarefa geradora (otimização)
<code>untied</code>	A tarefa pode ser retomada por qualquer thread do time (não precisa ser a que começou a executá-la). Padrão é <code>tied</code> (vinculada à mesma thread)
<code>priority(valor)</code>	Sugere prioridade de execução (valor numérico não-negativo; maior = recomendado executar antes)
<code>default(shared firstprivate none)</code>	Escopo padrão das variáveis dentro da tarefa
<code>private(list) / firstprivate(list) / shared(list)</code>	Mesmo conceito da seção 4, aplicado ao escopo da tarefa
<code>depend(in: list)</code>	Esta tarefa depende de tarefas anteriores que escreveram (<code>out / inout</code>) nessas variáveis
<code>depend(out: list)</code>	Esta tarefa produz dados nessas variáveis; tarefas futuras com <code>in / inout</code> sobre elas dependerão desta
<code>depend(inout: list)</code>	Combina leitura e escrita — depende de qualquer tarefa anterior (<code>in</code> , <code>out</code> ou <code>inout</code>) sobre a variável, e futuras tarefas dependerão desta

7.3 Regras de escopo padrão em tasks

Situação	Escopo padrão
Variável estática ou global	shared
Variável local (automática) declarada fora da task	firstprivate (se a task for "órfã", sem task-pai)
Variável local declarada dentro do bloco task	private
Variável static declarada dentro do bloco task	shared
Objeto alocado dinamicamente (malloc , new) apontado de fora	shared (o ponteiro pode ser firstprivate, mas o conteúdo apontado é compartilhado)
Task com task-pai	Herda os atributos de compartilhamento da tarefa pai

7.4 Sincronização de tarefas

Diretiva	O que faz
#pragma omp taskwait	A tarefa atual espera suas tarefas-filhas diretas terminarem (não espera netos/descendentes mais profundos)
#pragma omp taskgroup	Espera todas as tarefas descendentes (filhas e seus descendentes) dentro do bloco terminarem — sincronização mais profunda que taskwait . Pode ter cláusula task_reduction
#pragma omp taskyield	Ponto explícito onde a tarefa atual pode ser suspensa, liberando a thread para executar outra tarefa (evita bloqueios)
Barreira implícita (barrier , fim de parallel)	Garante que todas as tarefas criadas por qualquer thread do time terminem

7.5 taskloop — paralelizar laços via tasks

```
#pragma omp taskloop [simd] [cláusulas]
for (...) { ... }
```

Cláusula	Efeito
grainsize(n)	Cada bloco (chunk) de tarefa terá entre n e 2n iterações
num_tasks(n)	Cria exatamente n tarefas para cobrir todas as iterações

Cláusula	Efeito
<code>nogroup</code>	Por padrão, <code>taskloop</code> cria uma região <code>taskgroup</code> implícita; essa cláusula remove essa barreira
<code>collapse(n)</code>	Mesmo conceito da seção 3, aplicado a laços aninhados dentro do <code>taskloop</code>
Também aceita: <code>shared</code> , <code>private</code> , <code>firstprivate</code> , <code>lastprivate</code> , <code>default</code> , <code>final</code> , <code>if</code> , <code>untied</code> , <code>mergeable</code> , <code>depend</code> , <code>priority</code>	Mesmo significado das seções 4 e 7.2

`taskloop simd` : além de dividir em tarefas, aplica vetorização SIMD dentro de cada bloco.

8. Funções de Biblioteca (Runtime API)

Função	O que retorna/faz
<code>omp_get_num_threads()</code>	Número de threads no time atual (só é significativo dentro de uma região paralela)
<code>omp_get_thread_num()</code>	O <code>id</code> (tid) da thread que chama a função, de 0 a N-1
<code>omp_get_num_procs()</code>	Número de processadores/núcleos disponíveis na máquina
<code>omp_in_parallel()</code>	Retorna se o código está atualmente dentro de uma região paralela ativa
<code>omp_get_wtime()</code>	Retorna o tempo (em segundos, <code>double</code>) desde uma referência fixa — usado para medir tempo de execução
<code>omp_set_num_threads(n)</code>	Define o número de threads a usar nas próximas regiões paralelas
<code>omp_init_lock(&lock)</code>	Inicializa uma trava (lock) manual
<code>omp_set_lock(&lock)</code>	Adquire a trava (bloqueia se outra thread já a possui)
<code>omp_unset_lock(&lock)</code>	Libera a trava

9. Variáveis de Ambiente

Variável	Efeito
<code>OMP_NUM_THREADS</code>	Define o número padrão de threads (sobreposta por <code>num_threads()</code> no código)

Variável	Efeito
OMP_SCHEDULE	Define o tipo/chunk de <code>schedule</code> usado quando a cláusula do código é <code>schedule(runtime)</code>
OMP_STACKSIZE	Tamanho da pilha de cada thread
OMP_PLACES	Define o mapeamento de threads para localizações físicas (núcleos, sockets)
OMP_PROC_BIND	Controla se/como as threads ficam "fixadas" a processadores específicos

10. Compilação

```
gcc -fopenmp arquivo.c -o programa # habilita suporte a OpenMP no GCC
./programa                        # executa
OMP_NUM_THREADS=4 ./programa      # define threads via variável de
ambiente
```

11. Resumo mental rápido

- `parallel` → cria threads. `for` → divide iterações. `parallel for` → os dois juntos.
- `single` → uma thread qualquer, com barreira. `master` → sempre a thread 0, sem barreira.
- `shared` → risco de condição de corrida. `private` → cópia isolada, não inicializada. `firstprivate` → cópia isolada, inicializada com valor de fora.
- `static` → previsível, baixo overhead. `dynamic` → adaptável, mais overhead. `guided` → híbrido.
- `atomic` → protege 1 operação simples. `critical` → protege um bloco inteiro (mais caro).
- `task` → unidade de trabalho independente, útil fora do padrão de laço regular (recursão, listas, grafos de dependência via `depend`).
- `taskwait` → espera só filhos diretos. `taskgroup` → espera toda a descendência.